A N A L Y T I C S H U B

Role-Based Qlik Dashboard Portal

Technical Blueprint & Product Specification

Next.js · Python (FastAPI) · Qlik Embed · JWT · Vercel · Render

2025 — Corporative Internal Document

Executive Summary

This document defines the architecture, design principles, and implementation roadmap for a corporate Analytics Hub — a role-aware web portal that unifies all Qlik Sense dashboards across every business unit of the corporation into a single, branded, intelligent interface.

Today, business users navigate multiple Qlik spaces, memorize URLs, and cannot easily discover dashboards outside their immediate team. The Analytics Hub solves this by providing:

- **Search-first experience:** A Google-style smart search bar as the primary entry point.
- **Role-aware home screen:** A personalized tile grid showing the reports most relevant to each user's role.
- **Embedded analytics:** Inline Qlik iframe embeds with SSO, so users never leave the portal.
- **Governance:** Fine-grained access governed by Active Directory / SSO groups.
- **Minimalist UI:** A clean, minimal aesthetic inspired by modern productivity tools like Claude.ai, Notion, and Linear.

The platform builds on existing infrastructure: a Python/FastAPI backend deployed on Render and a Next.js frontend on Vercel, minimizing greenfield effort.

01 Problem Statement & Vision

Why this portal, and what it must become

1.1 The Problem Today

Across the corporation, each subsidiary or business unit maintains its own Qlik space. Users face three friction points:

- Report fragmentation — dashboards exist in at least 6 separate Qlik spaces. Users do not know where to look.
- No role filtering — every user entering a Qlik space sees all apps, even ones irrelevant to their job.
- Lack of context — a Qlik app tile shows only a name and thumbnail. There is no description, owner, update frequency, or data source information.
- No single sign-on to the portal level — users must authenticate per space.

1.2 The Vision

The Analytics Hub is a lightweight, fast, beautiful web application that acts as the "front door" to all corporate analytics. When a user logs in, they see exactly what they need — no more, no less — presented in a clear visual grid with rich metadata, searchable in under two keystrokes.

The UX reference is deliberate: the same comfortable minimalism used by Claude.ai, where a single centered search bar anchors the page and content radiates outward from intent, not from hierarchy.

02 Core Functionalities

Every capability the portal must deliver

2.1 Authentication & Role Management

	SSO / OAuth 2.0 Login	Users authenticate via “Request of a 8 digits Code” (10 min validy) to be send from noreply@unilinkportal.com using Resend. Only emails from unilinktransportation.com	**Must Have**
	Role-Based Access Control	Each user belongs to one or more roles (e.g., Finance Analyst, Supply Chain Manager, Executive). Roles map to visible report sets.	**Must Have**
	Qlik JWT Ticket Integration	Backend generates Qlik-compatible JWT tickets on behalf of the user for transparent embed authentication — user is never prompted to log into Qlik separately.	**Must Have**
	Row-Level Security Pass-through	Backend injects user attributes (company, region, cost center) into the JWT so Qlik RLS rules apply automatically inside each embedded report.	**Must Have**

2.2 Home Screen — Search-First Layout

The home screen replicates the interaction model of a modern AI assistant interface, adapted for report discovery:

Zone A	Top center: Corporate logo + user avatar + notification bell
Zone B	Center: Large, rounded search bar (placeholder: "Search reports, KPIs, departments…")
Zone C	Below search: "Pinned" row — the user's 4–6 most accessed reports as large icon cards
Zone D	Below pinned: Categorized grid — cards organized by domain (Finance, Operations, HR, etc.)

Zone E	Bottom: Quick-access footer links (Admin, Help, Changelog)

2.3 Smart Search Engine

	Full-text Search	Searches report title, description, tags, data sources, and owner name in real time as the user types.	**Must Have**
	Instant Results Dropdown	Results appear in a floating panel within 150ms, showing report icon, name, category, and a one-line description.	**Must Have**
	Semantic / AI Search (Phase 2)	Embeddings-based search via the Anthropic API or OpenAI so queries like "monthly revenue by region" surface the right report even if the words do not match.	**Nice to Have**
	Tag & Filter Chips	Users refine search results by category, business unit, data freshness, or favorite status using one-click chips below the search bar.	**Must Have**
	Trending & Recent	When the search bar is focused but empty, show "Recently viewed" and "Trending this week" cards drawn from aggregated access logs.	**Should Have**

2.4 Report Card Design

Each report in the grid is represented by a card. Cards are the primary visual unit of the interface and must communicate maximum information at a glance:

Card Element	Description
Report Icon / Thumbnail	A branded icon (auto-generated from category color + initials) or a Qlik app screenshot thumbnail
Report Title	Bold, 16px, truncated at 2 lines
Category Badge	Color-coded pill: Finance (blue), HR (teal), Supply Chain (orange), etc.
One-line Description	Plain language: what question does this dashboard answer?
Data Freshness	"Updated 3 hours ago" — pulled from Qlik reload schedule metadata

Owner Name	The business owner responsible for the data, with avatar
Favorite Star	User can pin to their personal top row
Quick-view Button	Hover reveals a small eye icon — opens a 400px preview panel without leaving the home screen
Open Full Screen	Primary CTA — opens the Qlik embed in a full modal or dedicated route

2.5 Embedded Report Viewer

🖥️	**Full-page Embed Mode**	Report opens in a route (e.g. /reports/[id]) with the Qlik app occupying 100% of the viewport minus a slim header bar.	**Must Have**
🗂️	**Sidebar Drawer Mode**	From search results, a report can open in a right-side drawer (60% wide) so the user keeps the home screen context visible.	**Should Have**
🔗	**Deep-link Sharing**	Each embedded view has a stable URL the user can copy, share with colleagues, or bookmark. Access is still gated by role.	**Must Have**
↩️	**Back Navigation**	Breadcrumb trail: Home → Category → Report Name. Single click returns to the previous state.	**Must Have**

2.6 Admin Console

📋	**Report Registry**	Admins manage the catalog: add/edit/archive report cards, assign categories, write descriptions, set role visibility.	**Must Have**
👔	**Role Manager**	Create and edit roles, assign users or AD groups, preview what a given role sees.	**Must Have**
📊	**Usage Analytics**	Dashboard showing most-viewed reports, peak usage hours, search queries with no results (discovery gaps).	**Should Have**

🔔	**Alert & Annotation System**	Admins can attach a banner to a report card: "Data issue detected — use with caution" or "New fiscal year data loaded."	**Nice to Have**

03 UX & Design System

Visual language, layout principles, and interaction patterns

3.1 Design Philosophy

The portal adopts Calm Technology principles: it provides information without demanding attention, and recedes when not needed – ONLY LIGHT mode (block any Dark mode from the browser and other similar). The aesthetic target is the intersection of:

- Claude.ai — centered search bar, airy whitespace, no sidebar clutter
- Linear.app — sharp typography, fast interactions, keyboard-first
- Notion — card-based content organization with rich metadata
- Apple Tablet UI — generous touch targets, readable at a distance

3.2 Color System

Token	Value & Usage
Background	#FFFFFF / #F9FAFB — white with very subtle warm gray for cards
Primary Navy	#1B3A5C — headers, primary text, logo
Accent Blue	#2563EB — interactive elements, search focus ring, active states
Category — Finance	#1D4ED8 (dark blue pill)
Category — HR	#0D9488 (teal pill)
Category — Ops	#D97706 (amber pill)
Category — Sales	#7C3AED (violet pill)
Category — Executive	#1B3A5C (navy pill)
Text Primary	#111827
Text Secondary	#6B7280
Dividers	#E5E7EB

3.3 Typography

Role	Specification

Display / Hero	Inter 700 — 32–40px — Used for page titles and hero text
Section Heading	Inter 600 — 20–24px — Used for category group labels
Card Title	Inter 600 — 15–16px — Used inside report cards
Body / Description	Inter 400 — 14px — Used for descriptions and metadata
Badge / Label	Inter 500 — 11–12px uppercase — Used for category pills and status tags
Code / Monospace	JetBrains Mono 400 — 13px — Used for IDs, API keys, debug info

3.4 Grid & Spacing

- Base unit: 4px. All spacing values are multiples of 4 (8, 12, 16, 24, 32, 48, 64, 96).
- Card grid: CSS Grid with auto-fill columns, min 280px, max 340px, gap 24px.
- Page max-width: 1280px centered with 48px horizontal padding on desktop; full-bleed on mobile.
- Search bar: 640px max-width, centered, 52px height, 16px corner radius, subtle shadow on focus.
- Cards: 16px corner radius, 1px border (#E5E7EB), 8px shadow on hover (0 4px 24px rgba(0,0,0,0.08)).

3.5 Motion & Interaction

- All transitions: 150ms ease-out (fast enough to feel instant, long enough to feel smooth).
- Search dropdown: slides down with a subtle scale transform (0.98 → 1.0).
- Card hover: lifts with a 4px translateY and shadow intensification.
- Page transitions: Next.js App Router shared layouts with Framer Motion layout animations.
- Loading states: skeleton screens (not spinners) for card grids; shimmer animation at 1.5s.
- Keyboard navigation: full arrow-key navigation in search results; Escape to close modals.

04 Technical Architecture & Stack

Recommended technologies building on existing infrastructure

4.1 Stack Decision Matrix

Layer	Technology	Rationale
Frontend	Next.js 14 (App Router)	Existing stack. Server components reduce JS bundle; file-based routing aligns with /reports/[id] pattern.
Styling	Tailwind CSS + shadcn/ui	Utility-first + accessible headless components. No design-system overhead.
Animation	Framer Motion	Production-grade motion for layout animations and page transitions.
State	Zustand + React Query	Zustand for lightweight global UI state; React Query for server-state caching of report catalog.
Search UI	cmdk (Command Menu)	Accessible, keyboard-first command palette used by Linear and Vercel — perfect for search-first pattern.
Backend	FastAPI (Python)	Existing stack on Render. Handles JWT generation for Qlik, role resolution, and catalog API.
Auth	NextAuth.js + OAuth2	Integrates with Azure AD / Okta. JWT tokens carry role claims forwarded to backend.
Qlik Embed	@qlik/embed-web-components	Official Qlik web component library. Handles session management and responsive embed.
Database	PostgreSQL (AIVEN)	Report catalog, user preferences, favorites, access logs. AIVEN adds realtime and row-level security.
Search Index	Typesense Algolia	Sub-10ms full-text search. Typesense is free and Docker-deployable on Render.
AI Search (P2)	Anthropic Claude API + pgvector	Semantic search via embeddings stored in PostgreSQL with the pgvector extension.
Hosting	Vercel (front) + Render (back)	Existing deployment targets — no migration needed.

4.2 High-Level Architecture Diagram

```
[ Browser / Corporate Tablet ]
```

```
                    |   HTTPS / NextAuth Session Cookie
                    [ Next.js App on Vercel ]
                      |   REST API Calls (JWT Bearer)
                  [ FastAPI Backend on Render ]
                    |                |                |
            [ PostgreSQL ]  [ Typesense ]  [ Qlik Cloud ]
             Catalog / Prefs   Search Index   Embed API + JWT
```

4.3 Qlik Embed Integration — Key Flow

The most critical technical integration is transparent Qlik authentication. Here is the recommended flow:

- User logs into the portal via NextAuth + Azure AD.
- Next.js calls POST /api/qlik/token on the FastAPI backend, passing the user's role claims.
- FastAPI signs a Qlik-compatible JWT (RS256) with the user's subject, groups, and custom attributes.
- The JWT is returned to the browser and stored in memory (not localStorage).
- The <qlik-embed> web component is initialized with the JWT and the app GUID from the report catalog.
- Qlik Cloud validates the JWT, applies RLS, and renders the app inside the iframe.

```python
// FastAPI — Qlik JWT generation (Python)
import jwt, time

def generate_qlik_jwt(user_sub: str, groups: list[str], attrs: dict) -> str:
    payload = {
        "sub":    user_sub,
        "name":   attrs.get("display_name"),
        "groups": groups,
        "jti":    str(uuid.uuid4()),
        "iat":    time.time(),
        "exp":    time.time() + 3600,
        "iss":    settings.QLIK_ISSUER,
        **attrs   # company, region, cost_center …
    }
    return jwt.encode(payload, settings.QLIK_PRIVATE_KEY, algorithm="RS256")
```

```typescript
// Next.js — Embedding the Qlik app (TSX)
import "@qlik/embed-web-components";

export default function ReportViewer({ appId, sheetId, qlikToken }) {
  return (
```

```
      <qlik-embed
        ui="classic/app"
        app-id={appId}
        sheet-id={sheetId}
        auth-type="Jwt"
        jwt={qlikToken}
        style={{ width: "100%", height: "calc(100vh - 64px)" }}
      />
    );
  }
```

05 AI-Powered Semantic Search

Using embeddings to make report discovery truly intelligent (most wanted as RAG base)

5.1 Why Semantic Search for Reports

Users rarely know the exact title of a dashboard. They think in business terms: "I want to see how much we sold in Mexico last month" — not "Revenue_MX_Monthly_v3". Keyword search fails here. Semantic search using vector embeddings bridges this gap.

5.2 Implementation Approach

Component	Detail
Embedding Model	Anthropic's embeddings API — both produce 1536-dimensional vectors at low cost – use as RAG knowledge base
Storage	pgvector extension on PostgreSQL — each report card's description + tags are embedded and stored as a vector
Query Flow	User's search query → embed via API → cosine similarity search in pgvector → return top-K reports
Hybrid Search	Combine keyword (BM25 via Typesense) and semantic (pgvector) scores with a weighted sum — best of both worlds
Re-ranking	Optionally pass top-20 candidates to Claude API for intelligent re-ranking based on user role context
Cost	~$0.0001 per query at 1000 users/day = ~$3/month — negligible

5.3 Natural Language Report Summary (Enhancement)

When a user hovers a card or opens the quick-view drawer, the portal can call the Claude API to generate a one-paragraph plain-language summary of what the report shows, what decisions it supports, and when data was last refreshed — pulling metadata from the catalog and Qlik's reload API.

```
# FastAPI — Generate natural language report summary
async def generate_report_summary(report: ReportMetadata) -> str:
    prompt = f"""
    You are a data analyst assistant. Given this report metadata:
    Title: {report.title}
    Category: {report.category}
    Description: {report.description}
```

```
    Data sources: {report.data_sources}
    Last refreshed: {report.last_reload}

    Write a 2-sentence plain-language summary for a business user.
    Focus on: what question it answers, and who should use it."""

    response = anthropic_client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=200,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text
```

06 Data Model & API Design

PostgreSQL schema and FastAPI endpoint surface

6.1 Core Tables

```
-- Reports catalog
CREATE TABLE reports (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  qlik_app_id   TEXT NOT NULL,
  qlik_sheet_id TEXT,
  title         TEXT NOT NULL,
  description   TEXT,
  category      TEXT,
  tags          TEXT[],
  owner_name    TEXT,
  data_sources  TEXT[],
  last_reload   TIMESTAMPTZ,
  embedding     vector(1536),   -- pgvector for semantic search
  is_active     BOOLEAN DEFAULT TRUE,
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Role → Report mapping
CREATE TABLE role_report_access (
  role_name  TEXT NOT NULL,
  report_id  UUID REFERENCES reports(id),
  PRIMARY KEY (role_name, report_id)
);

-- User preferences
CREATE TABLE user_preferences (
  user_sub        TEXT PRIMARY KEY,
  pinned_reports  UUID[],
  recent_reports  UUID[],
  theme           TEXT DEFAULT 'light'
);

-- Access log (for trending & usage analytics)
CREATE TABLE access_log (
  id         BIGSERIAL PRIMARY KEY,
  user_sub   TEXT,
  report_id  UUID REFERENCES reports(id),
  accessed_at TIMESTAMPTZ DEFAULT NOW()
);
```

6.2 REST API Endpoints

Endpoint	Purpose
GET /api/reports	Returns role-filtered report catalog for the authenticated user
GET /api/reports/{id}	Full metadata for a single report including AI summary
GET /api/reports/search	Hybrid full-text + semantic search. Query params: q, category, tags
GET /api/reports/trending	Top reports by access count in the last 7 days, filtered by role
POST /api/qlik/token	Generates a signed Qlik JWT for the current user session
GET /api/user/preferences	Returns pinned reports, recent history, theme preference
PATCH /api/user/preferences	Updates user preferences (pin/unpin, theme)
GET /api/admin/reports	Admin: full unfiltered catalog with edit controls
POST /api/admin/reports	Admin: create a new report card in the catalog
PATCH /api/admin/reports/{id}	Admin: update report metadata
DELETE /api/admin/reports/{id}	Admin: soft-delete (sets is_active = false)
GET /api/admin/usage	Admin: usage analytics — views per report, search queries, gaps

07 Implementation Roadmap

Phased delivery plan from MVP to intelligent platform

Phase 1 — MVP (Weeks 1–6)

Wk	Milestone	Deliverables
1–2	**Auth + Skeleton**	NextAuth integration with Azure AD, JWT role extraction, FastAPI /api/reports returning mock data, basic Next.js layout with header and grid placeholder
3	**Report Catalog**	PostgreSQL schema deployed on Render, admin seeding script, GET /api/reports with role filtering, report cards rendering in the grid
4	**Qlik Embed**	Qlik JWT flow end-to-end, <qlik-embed> component in /reports/[id], SSO working with test user
5	**Search**	Typesense index populated from catalog, search bar with dropdown results, filter chips by category
6	**Polish + QA**	Favorites, pinned row, mobile responsive layout, loading skeletons, accessibility audit (WCAG AA), staging deployment

Phase 2 — Intelligence (Weeks 7–12)

- Semantic search via pgvector embeddings
- Claude API integration for natural language report summaries on hover
- Trending reports surfaced from access_log analytics
- Admin usage dashboard
- Report alert banners (admin-authored)
- Notification system for new report availability by role

Phase 3 — Scale & Governance (Weeks 13–20)

- Qlik reload webhook → automatic last_reload metadata sync
- Request access flow: users can request access to reports outside their role
- Multi-language UI (i18n with next-intl)
- SSO to additional IdPs (Google Workspace, Okta)
- Performance optimization: edge caching of catalog on Vercel CDN
- Audit trail: every report view logged with timestamp and user for compliance

08 Security & Governance

Protecting sensitive corporate analytics data

8.1 Security Controls

	JWT Expiry & Rotation	Qlik JWTs expire after 60 minutes. The frontend silently refreshes via /api/qlik/token before expiry using a background timer.	**Critical**
	API Rate Limiting	FastAPI applies rate limiting (slowapi) per user_sub: 300 req/min standard, 10 req/min for token generation endpoint.	**Critical**
	CORS Policy	Backend CORS restricted to the Vercel deployment origin. No wildcard origins permitted.	**Critical**
	Secret Management	Qlik private key, database URL, and API keys stored in Render environment secrets and Vercel environment variables. Never committed to source control.	**Critical**
	Audit Logging	Every access_log entry captures user_sub, report_id, IP address (hashed), timestamp, and session ID for compliance review.	**Required**
	Content Security Policy	Next.js headers configuration enforces strict CSP, allowing iframes only from your Qlik Cloud tenant domain.	**Required**

8.2 Data Governance

The portal does not store any Qlik data or visualizations. All data remains within Qlik Cloud and is rendered only inside authenticated iframe sessions. The portal's PostgreSQL database stores only catalog metadata (titles, descriptions, tags) — never underlying business data.

Row-level security within Qlik is the authoritative data access control. The portal's role system controls which apps are visible and launchable, but cannot grant access to data that Qlik RLS denies. This dual-layer model ensures defense in depth.

09 Frontend Component Architecture

Next.js App Router component tree and key page structure

9.1 Page Routes

```
app/
├── layout.tsx                  # Root layout: providers, auth, header
├── page.tsx                    # Home: search bar + report grid
├── reports/
│   └── [id]/page.tsx           # Full-screen Qlik embed viewer
├── admin/
│   ├── layout.tsx              # Admin guard + sidebar
│   ├── page.tsx                # Admin dashboard: usage stats
│   ├── reports/page.tsx        # Report catalog CRUD
│   └── roles/page.tsx          # Role management
├── api/
│   ├── auth/[...nextauth]/     # NextAuth handlers
│   └── proxy/route.ts          # Backend proxy (avoids CORS from browser)
└── (auth)/
    └── login/page.tsx          # Login page
```

9.2 Key Components

Component	Responsibility
<SearchBar />	cmdk-based command palette. Fetches /api/reports/search on input. Shows trending when focused+empty.
<ReportGrid />	React Query + CSS Grid. Groups cards by category. Supports virtualization (react-virtual) for 200+ cards.
<ReportCard />	Individual card with icon, title, category pill, description, metadata footer. Hover → QuickPreview.
<QuickPreview />	Radix UI Sheet (drawer) containing a Qlik single-sheet embed at 50% width.
<QlikEmbed />	Wrapper around <qlik-embed> web component. Handles token injection and resize observation.
<RoleGuard />	HOC that checks the user's session roles against the required role list. Redirects on failure.
<AdminTable />	TanStack Table v8 with sorting, filtering, and inline edit for report catalog management.
<UsageChart />	Recharts BarChart showing daily views per report for the past 30 days.

10 Success Metrics & KPIs

How to measure the portal is delivering value

KPI	Target
Time-to-first-report	Target: <15 seconds from login to viewing a Qlik dashboard (vs. current ~3 minutes navigating Qlik spaces)
Search-to-result rate	Target: >85% of searches return a click within the first 5 results
Report discovery	Target: Average user accesses 3x more unique reports vs. pre-portal baseline
Abandonment rate	Target: <10% of users who open the portal leave without viewing a report
Admin catalog coverage	Target: 100% of active Qlik apps registered in catalog within 60 days of launch
Embed load time	Target: Qlik iframe fully interactive in <4 seconds on corporate network
User satisfaction (CSAT)	Target: >4.2/5 in quarterly internal UX survey
Zero auth complaints	Target: 0 tickets per week about being prompted to log into Qlik separately

Appendix — Reference Links & Further Reading

Resource	URL
Qlik Embed Web Components	https://qlik.dev/embed/web-components/
Qlik JWT Authentication	https://qlik.dev/authenticate/jwt/
Next.js App Router Docs	https://nextjs.org/docs/app
FastAPI Documentation	https://fastapi.tiangolo.com/
NextAuth.js	https://next-auth.js.org/
cmdk — Command Menu	https://cmdk.paco.me/
Framer Motion	https://www.framer.com/motion/
Typesense Search	https://typesense.org/docs/
pgvector Extension	https://github.com/pgvector/pgvector
shadcn/ui Components	https://ui.shadcn.com/
TanStack Query (React Query)	https://tanstack.com/query/latest
Zustand State Manager	https://zustand-demo.pmnd.rs/
Anthropic API Documentation	https://docs.anthropic.com/

Document version 1.0 — Prepared for internal architecture review. All implementation details are subject to refinement during discovery sessions with the Qlik administrator and security team.